

AI in mobile apps

On-device ML, LLMs, RAG and MCP

Speaker

Dinu-Ștefan Rusu

Date

2026

Affiliation

CodingShadows · rusudinu.ro

Who is talking

Dinu-Ștefan Rusu

Founder and senior engineer at CodingShadows. I build mobile products for clients and teach mobile development at university.

Where to find me

rusudinu.ro · codingshadows.com

Building for next-gen mobile experiences

Flutter and native apps in production, plus the machine learning parts that sit behind them.

What this talk covers

On-device ML

ML Kit text recognition and barcodes. What the phone does with no network at all.

Tools and MCP

Function calling, Model Context Protocol servers, three notebooks.

Talking to a model

Hosted or local, the request shape, streaming, and where the key may live.

Security

Input checks, token budgets, human approval, output guardrails.

Retrieval

Embeddings, distance, approximate search, chunking, one real index.

1

The camera is already a sensor

ML Kit text recognition, barcodes, and what never leaves the phone

Contents		Page
1	The camera is already a sensor	4
2	A chat box is a second user interface	11
3	Giving the model your own data	18
4	Letting the model act on your app	27
5	Nothing in this chain is trusted by default	32

What runs on the phone

Text recognition

Latin, Chinese, Japanese, Korean and Devanagari scripts. Returns blocks, lines and elements with bounding boxes.

Barcode scanning

Most one- and two-dimensional formats. Parses known payloads such as URLs, contacts and Wi-Fi.

Face and pose

Landmarks, contours and body keypoints at frame rate, with no server and no upload.

Image labeling

Generic labels from a bundled model, or your own model exported to TensorFlow Lite.

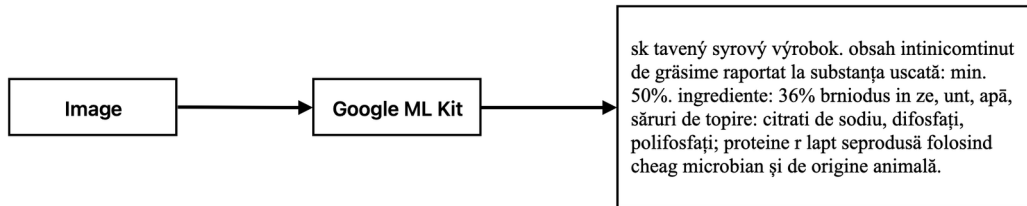
Translation

Small language packs downloaded once, then translation with the network off.

Document scanner

Edge detection, perspective correction and a clean page image handed back to the app.

Text recognition returns text, not meaning



The model reads a food label and hands back one string: broken words, missing diacritics, wrong characters, no structure. The app wanted a list of additive codes.

Recognition is the cheap half of the job. Everything after it is ordinary code that you write, test and own.

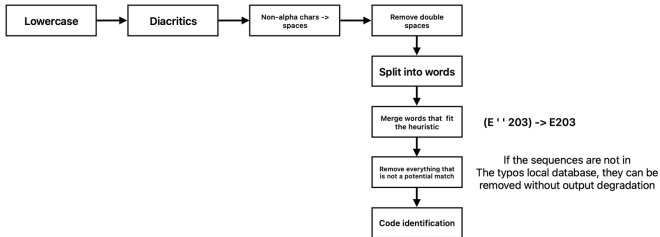
Reading text from a camera frame

```
val options = TextRecognizerOptions.DEFAULT_OPTIONS
val recognizer = TextRecognition.getClient(options)

val image = InputImage.fromMediaImage(mediaImage, rotationDegrees)
recognizer.process(image)
    .addOnSuccessListener { result ->
        val lines = result.textBlocks.flatMap { it.lines }
        handle(lines.map { it.text })
    }
    .addOnFailureListener { error -> report(error) }
```

The same call exists on iOS and through the Flutter plugin. Close the image when you are done, or the camera pipeline stalls after a few frames.

From raw text to additive codes



Lowercase, strip diacritics, replace every non-letter with a space, split, then merge fragments that fit the shape of a code. What is left is matched against a local list.

Normalize first, match second. A short deterministic pipeline recovers most of what the recognizer got wrong.

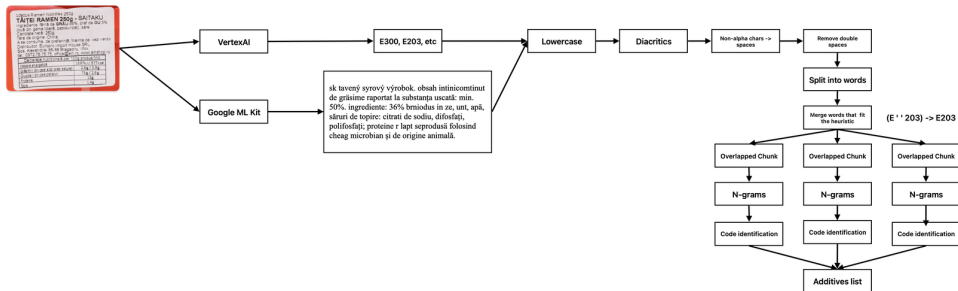
On the device or in the cloud

Sent to a hosted multimodal model, the same photo comes back as the finished list of codes. One call replaces the whole pipeline above.

Question	On device	Cloud	What decides it
Latency	tens of ms	hundreds of ms	a camera preview needs the device
Works offline	yes	no	shops and basements have no signal
Cost per call	zero	per token	a scanner runs constantly
Privacy	nothing leaves	image uploaded	pictures of documents and people

Use both. Recognize on the device, escalate to the cloud only when the local result is empty.

The whole pipeline on one slide



One photo, two recognizers, one normalization chain, one local lookup. Both branches return the same shape of result, so the rest of the app does not care which answered.

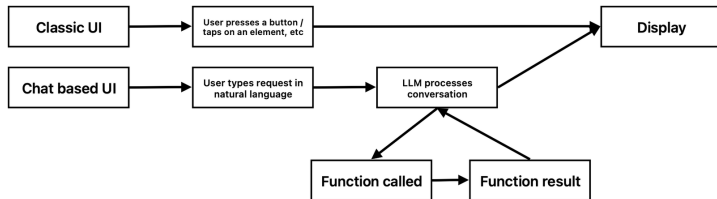
2

A chat box is a second user interface

Hosted or local, the request shape, streaming, and where keys live

Contents		Page
1	The camera is already a sensor	4
2	A chat box is a second user interface	11
3	Giving the model your own data	18
4	Letting the model act on your app	27
5	Nothing in this chain is trusted by default	32

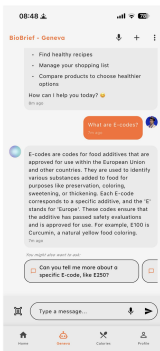
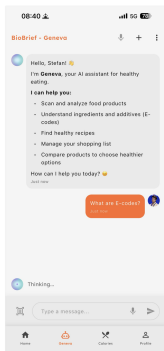
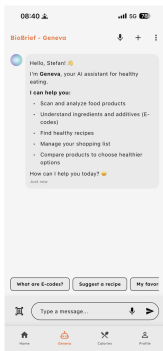
Chat is a second way into the same app



The classic path is a tap that calls a function and draws a screen. The chat path is a sentence that the model turns into the same call. Both end in the same display code.

You are not building a chatbot. You are adding a second entry point to functions your app already has.

What it looks like on the phone



Say what it can do.

The first message lists the capabilities. Users do not guess what a text box accepts.

Offer starters.

Three suggested questions remove the empty-box problem and teach the vocabulary.

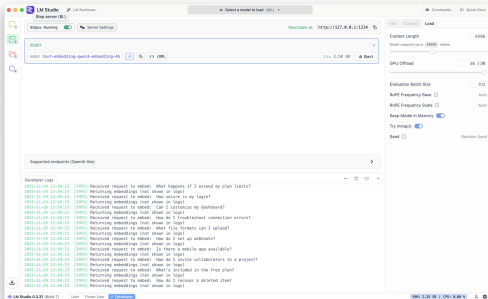
Follow-up chips keep the conversation inside the part of the app you tested.

Hosted model or local model

	Hosted API	Local runtime
Where it runs	a provider's servers	your laptop, or the phone
Quality	the strongest models available	smaller models, quantized
Cost	per token, every call	hardware you already bought
Data	leaves the device	never leaves the machine
Good for	production features	demos, tests, sensitive data

The API surface is the same. Both speak the OpenAI-compatible chat format, so the switch is a base URL and a model id.

One local endpoint, two models



LM Studio

A desktop app that downloads open-weight models and serves them over HTTP on `127.0.0.1:1234`.

Load one chat model and one embedding model. The server exposes both on the same OpenAI-compatible paths.

The developer log shows every request, which makes a failing retrieval step obvious.

The request shape

```
from openai import OpenAI

client = OpenAI(base_url="http://127.0.0.1:1234/v1", api_key="lm-studio")

response = client.chat.completions.create(
    model="qwen/qwen3.5-9b",
    messages=[{"role": "system", "content": SYSTEM_PROMPT},
              {"role": "user", "content": question}],
    temperature=0.2,
)
```

A list of messages in, one message out. The model has no memory between calls, so the whole conversation is sent again every turn. That is the bill and the context limit.

Streaming, and where the key lives

```
stream = client.chat.completions.create(  
    model="qwen/qwen3.5-9b", messages=messages, stream=True,  
)  
for chunk in stream:  
    piece = chunk.choices[0].delta.content  
    if piece:  
        emit(piece)
```

First token latency is what users call speed. Streaming turns one long wait into an answer that starts at once, and it costs three lines of code.

The key never ships in the app. Anything inside an installed binary is readable. Call your own backend, let it hold the provider key, and hand the app a short-lived user token.

3

Giving the model your own data

Embeddings, distance, approximate search, chunking, and a real index

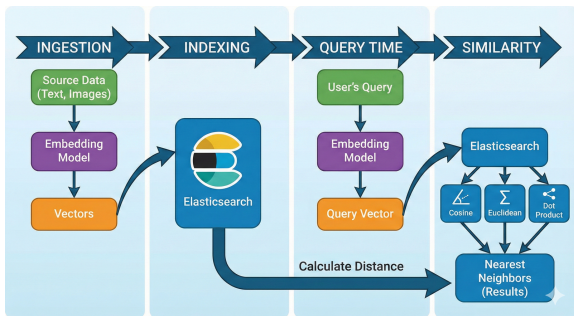
Contents		Page
1	The camera is already a sensor	4
2	A chat box is a second user interface	11
3	Giving the model your own data	18
4	Letting the model act on your app	27
5	Nothing in this chain is trusted by default	32

Why not just send everything

	Stuff the whole corpus	Retrieve, then generate
Request size	everything you own	the few passages that matter
Result	rejected, or slow and costly	fits the window easily
Accuracy	facts get lost in the middle	evidence sits next to the question
Updates	a new model or a fine-tune	write the document, index it
Auditing	no idea what was used	you can cite the source file

Retrieval-augmented generation is a search problem. The model is the last step. Most of the quality comes from what you put in front of it.

Meaning becomes geometry



An embedding is a list of numbers.

A few hundred to a few thousand floating-point values that place a piece of text in a space.

Texts about the same thing land close together, even with no shared words. Keyword search cannot do that.

Documents and questions must go through the same model, or the distances mean nothing.

Measuring distance, and finding neighbors

Metric	Measures	Notes
Cosine	the angle between vectors	ignores length, the safe default for text
Dot product	angle and length together	same ranking as cosine when normalized
Euclidean	straight-line distance	sensitive to magnitude, used for images

Exact search scores every vector and returns the true nearest neighbors, at a cost that grows with the corpus. Approximate search walks a graph of close vectors instead.

Pick the metric the model was trained with, and start exact. A small corpus needs no approximate index.

Hybrid search

Keyword search

Scores exact term overlap.
Unbeatable for product codes,
error numbers and names no
model has seen.

Vector search

Scores meaning. Finds the
paragraph that answers the
question in different words.

Fusion

Run both, then merge the ranked
lists by rank rather than by score,
since the scores are not
comparable.

Hybrid search fixes the failure each half has on its own. Vector search misses a serial number. Keyword search misses a paraphrase.

Chunking decides what you can retrieve

✂ Split on structure

Headings, paragraphs, list items.
A chunk that crosses a section boundary answers neither section well.

↔ Keep an overlap

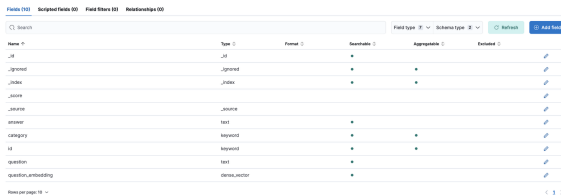
A sentence at the edge belongs to both neighbors. A small overlap stops answers from being cut in half.

📌 Carry the metadata

Store the source file, the section title and the access rules next to the vector, so an answer can be cited.

Chunk size is the main quality knob in a RAG system. Too large and the evidence is diluted. Too small and the passage loses its context.

The index, field by field



The screenshot shows the 'Fields' tab of the Elasticsearch Fields API. It displays a list of fields in the 'support_faq' index. The fields are: _id (type: _id, format: null, searchable: true, aggregatable: false, excluded: false), _ignored (type: _ignored, format: null, searchable: true, aggregatable: false, excluded: false), _index (type: _index, format: null, searchable: true, aggregatable: false, excluded: false), _score (type: _score, format: null, searchable: false, aggregatable: false, excluded: false), _source (type: _source, format: null, searchable: false, aggregatable: false, excluded: false), answer (type: text, format: null, searchable: true, aggregatable: false, excluded: false), category (type: keyword, format: null, searchable: true, aggregatable: true, excluded: false), id (type: keyword, format: null, searchable: true, aggregatable: true, excluded: false), question (type: text, format: null, searchable: true, aggregatable: false, excluded: false), and question_embedding (type: dense_vector, format: null, searchable: true, aggregatable: false, excluded: false). The table includes a search bar, filters for field type and schema type, and buttons for refresh and add field.

Name	Type	Format	Searchable	Aggregatable	Excluded
_id	_id		•		
_ignored	_ignored		•	•	
_index	_index		•	•	
_score	_score				
_source	_source				
answer	text		•		
category	keyword		•	•	
id	keyword		•	•	
question	text		•		
question_embedding	dense_vector		•		

A support FAQ index. `question` and `answer` are analyzed text, `category` is a keyword for filtering, and `question_embedding` is a dense vector.

The vector lives next to the text it came from. One query can filter, match keywords and compare vectors at once.

Asking the index a question

```
POST /faqs/_search
{
  "knn": {
    "field": "question_embedding",
    "query_vector": [0.00013, 0.02553, -0.00962, ...],
    "k": 3,
    "num_candidates": 100
  },
  "_source": ["question", "answer", "category"]
}
```

The query text is embedded by the same model that embedded the documents. Three passages come back with a score, and only those three go into the prompt.

RAG demo

Notebook

`demos/01_rag.ipynb`

1. Load a small corpus of notes. Markdown files about this talk and the models.
2. Embed and index them. An in-memory store built with the local embedding model.
3. Retrieve and read the distances. The retrieval step stays visible instead of hiding in a framework.
4. Build the prompt and answer. Answer only from the context, and cite the source file.

Every model call sits behind a flag that is off by default. The notebook runs end to end with no server.

4

Letting the model act on your app

Function calling, MCP servers, and three notebooks

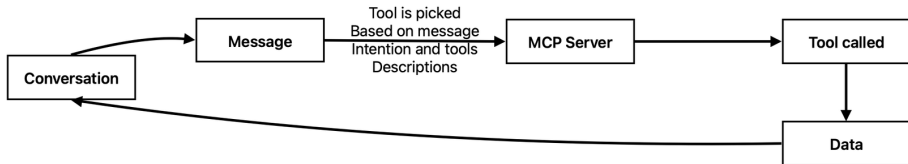
Contents		Page
1	The camera is already a sensor	4
2	A chat box is a second user interface	11
3	Giving the model your own data	18
4	Letting the model act on your app	27
5	Nothing in this chain is trusted by default	32

The tool calling loop

1. You send the tools with the message. A name, a description and a JSON schema for the arguments.
2. The model answers with a call, not with text. It picks a tool from the descriptions and fills in the arguments.
3. Your code runs the tool. The model never executes anything. It only asks.
4. The result goes back as another message. Tagged with the id of the call it answers.
5. The model writes the answer, or calls again. Cap the rounds, or a confused model will loop.

The description is the interface. The model chooses a tool by reading its description, so vague descriptions produce wrong calls.

MCP: one protocol between model and app



The Model Context Protocol standardizes that loop. A server advertises its tools, any compliant client discovers them, and the conversation continues with the result.

Write the integration once. Without MCP you write one tool adapter per client. With it you expose the app once.

Exposing your app as an MCP server

```
from mcp.server import MCPServer

server = MCPServer("recipes")

@server.tool()
def search_recipes(query: str, k: int = 3) -> str:
    """Search the recipe catalog and return matching titles."""
    return "\n".join(catalog.search(query, k))
```

Your existing service layer is already the tool. The wrapper is a name, a docstring and a typed signature over a function you shipped years ago.

MCP demo

Notebook	Needs	What it shows
<code>02_mcp.ipynb</code>	nothing	three tools, a real client session, discovery and calls over the protocol
<code>03_agent.ipynb</code>	local models, internet	retrieval as an MCP tool, live search, a safe page fetcher, one agent loop

Deterministic when grounding matters, agentic when coverage matters. One path always retrieves and then answers. The other advertises the tool and the model decides.

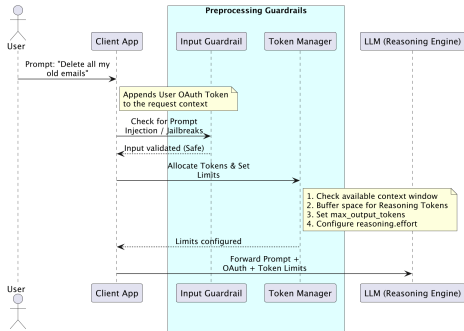
5

Nothing in this chain is trusted by default

Input checks, token budgets, human approval, output guardrails

Contents	Page
1 The camera is already a sensor	4
2 A chat box is a second user interface	11
3 Giving the model your own data	18
4 Letting the model act on your app	27
5 Nothing in this chain is trusted by default	32

Before the call: input check and budget



Attach the user, not a service account.

The request carries the signed-in user's token, so every tool call runs with that user's rights.

Check the prompt.

Screen for injection and jailbreak patterns before the text reaches the model.

Then set the budget: context left, space reserved for reasoning, a hard cap on output.

Reasoning, validation, human in the loop

Reasoning is not free

Planning tokens are generated before the answer and spend the same budget. Reserve space for them.

Validate every call

The server checks the token scope and the argument schema before anything runs.

Stop for consent

Deletes, payments and messages pause and ask the user before they happen.

The model proposes. Your server decides. Treat a tool call like a request arriving from a browser you do not control.

Output guardrails



Check for leaks

The answer can repeat context the current user was never allowed to see. Scan it before display.



Check the shape

If the app expects structured output, parse it and reject what does not fit instead of rendering it.



Check the citations

Verify that a cited source was actually in the retrieved set. An invented citation is a confident lie.

Guardrails sit on both ends of the call. One before the model sees the prompt, one before the user sees the answer.

One thing to take away

1. Most of the work is ordinary engineering. Recognition, normalization, search and validation are code you write and test.
2. Retrieval decides the quality, not the model. Chunk size, metric and metadata move the answer more than a bigger model does.
3. Every tool call is untrusted input. Scope it, validate it, and ask the user before anything irreversible.

Demos and slides

rusudinu.ro · codingshadows.com

Questions?

rusudinu.ro · codingshadows.com